try-with-resources and Suppressed Exceptions

8 min read · Updated 28 Jul 2026 · Topic 29 of 65

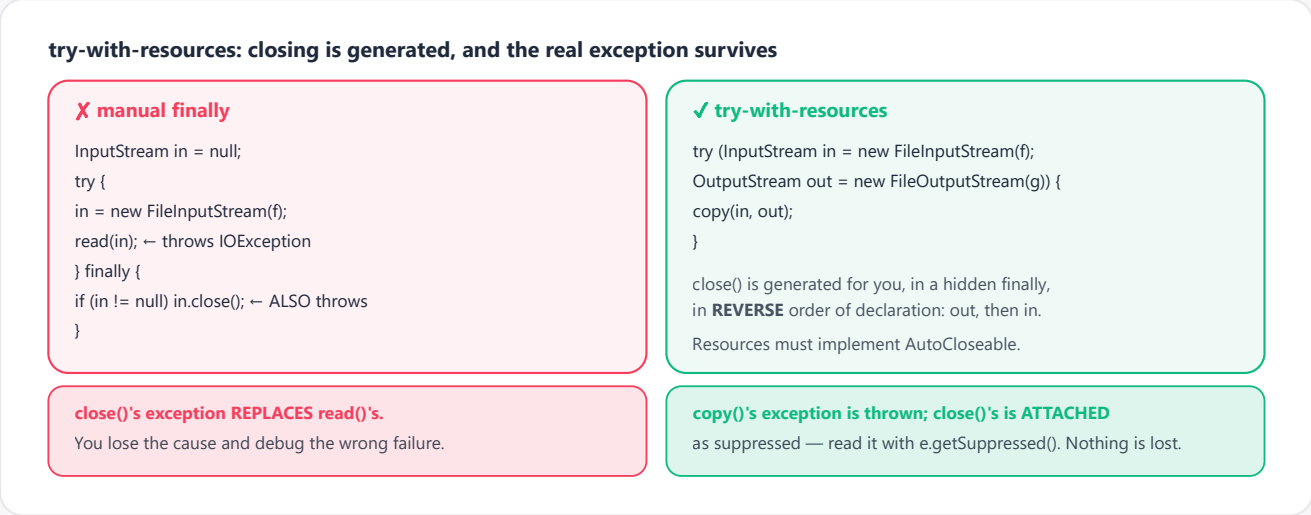
Bookmark · Mark done

Why it matters

Before Java 7, closing resources correctly took about nine lines of nested `try` / `finally` that almost nobody wrote properly — and getting it wrong either leaked file handles and connections or destroyed the exception that told you what really failed. try-with-resources reduces it to one line and fixes a problem most people did not know they had. The suppressed-exception mechanism it introduced is the part interviewers use to separate people who use the syntax from people who understand it.

The concept, simply

Declare resources in parentheses after `try`, and the compiler generates the `close()` calls for you — in a hidden `finally`, in reverse order of declaration, and without losing the original exception.



The bug on the left is easy to miss and expensive: when both the body and close() fail, the manual version reports the cleanup failure and silently loses the real cause.

Any resource must implement `AutoCloseable` — a single-method interface added in Java 7. `Closeable` (from `java.io`) extends it and narrows `close()` to throw only `IOException`.

How it works

Three guarantees, and all three matter:

1. `close()` **is always called**, on every exit path, exactly as a `finally` would.
2. **Resources close in reverse order of declaration** — the last opened is the first closed. That is the correct order when later resources wrap earlier ones, as a `BufferedReader` wraps a `FileReader`.
3. **The body's exception wins.** If the body throws *and* `close()` throws, the body's exception propagates and `close()`'s is attached to it as a *suppressed* exception, retrievable with `getSuppressed()`. Nothing is silently lost.

That third point is the reason the feature exists. In a hand-written `finally`, an exception from `close()` *replaces* the one from the body — so a failure to flush a buffer masks the real error that caused it.

Code examples

Before and after:

JAVA

Copy

```java
// ✗ Java 6 style — and this is the version that is actually correct.
// Most hand-written code was simpler than this, and wrong.
InputStream in = null;
OutputStream out = null;
try {
    in = new FileInputStream(src);
    out = new FileOutputStream(dst);
    copy(in, out);
} finally {
    try {
        if (in != null) in.close();
    } finally {
        if (out != null) out.close();     // must still run if in.close()
```

```java
// ✓ Java 7+ — same guarantees, plus suppression, in three lines.
try (InputStream in = new FileInputStream(src);
     OutputStream out = new FileOutputStream(dst)) {
    copy(in, out);
}   // out.close() then in.close(), automatically
```

Suppressed exceptions, made visible:

JAVA Copy

```java
class Failing implements AutoCloseable {
    public void close() { throw new IllegalStateException("close failed"); }
}

try {
    try (Failing f = new Failing()) {
        throw new RuntimeException("body failed");
    }
} catch (Exception e) {
    System.out.println(e.getMessage());                    // body failed
    System.out.println(e.getSuppressed()[0].getMessage()); // close failed
}
// Both survive. With a manual finally, "body failed" would be gone and yo
// would spend the afternoon investigating the close() failure.
```

The Java 9 improvement people often miss:

JAVA Copy

```java
// Java 7/8 — a resource created elsewhere had to be reassigned to a
// new local just to be managed:
Connection conn = pool.get();
try (Connection c = conn) { … }          // clumsy

// Java 9+ — an effectively final variable can be used directly:
Connection conn = pool.get();
try (conn) {
```

Real-world shape, and a caveat worth knowing:

JAVA Copy

```java
try (Connection conn = dataSource.getConnection();
     PreparedStatement ps = conn.prepareStatement(SQL);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) process(rs);
}   // rs, then ps, then conn — exactly the right order

// ⚠ Files.lines() returns a Stream that HOLDS AN OPEN FILE HANDLE. It is
// AutoCloseable and must be closed, which is easy to forget because most
// streams need no closing:
try (Stream<String> lines = Files.lines(path)) {
    lines.filter(l -> !l.isBlank()).forEach(this::handle);
}
// Without the try-with-resources, the descriptor leaks until GC.
```

Under the hood

The compiler expands the construct into nested `try` / `finally` blocks with a temporary variable holding the primary exception. Roughly: if the body throws, remember that exception; then call `close()` inside its own `try`, and if *that* throws, call `addSuppressed` on the remembered exception rather than letting the new one escape. You can see the generated shape with `javap -c`, and it is the code nobody wrote by hand correctly.

`AutoCloseable` **versus** `Closeable` is a small but real interview question. `AutoCloseable.close()` declares `throws Exception`, so it can be implemented by anything. `Closeable` predates it, extends it, narrows the declaration to `IOException`, and is documented as idempotent — calling `close()` twice is harmless. `AutoCloseable` makes no such promise, though the javadoc strongly recommends implementations be idempotent.

Design guidance for your own resources: make `close()` idempotent, and do not let it throw if you can avoid it — an exception from `close()` is nearly always less interesting

a way that matters — flushing a buffered write, committing a transaction — that is a reason to make the flush or commit an explicit call in the body rather than a side effect of closing.

Common mistakes

- **Still writing manual `finally` blocks.** Use try-with-resources for anything `AutoCloseable`.
- **Forgetting `Files.lines()` and `Files.walk()` hold file handles.** They are `AutoCloseable` streams and must be closed.
- **Closing manually inside the block.** The generated `close()` still runs — hence the idempotency requirement.
- **Assuming declaration order is close order.** It is the reverse, deliberately.
- **Ignoring suppressed exceptions when logging.** Most frameworks print them, but a hand-rolled handler that only logs `getMessage()` loses them.
- **A `close()` that throws routinely.** It buries the real failure behind a cleanup error.
- **Declaring an unused resource just to close it (pre-Java 9).** On Java 9+ use `try (existingVar)`.

Interview lens

> **Q: "What is try-with-resources and what problem does it solve?"**
> A: It declares resources in parentheses after `try`, and the compiler generates the `close()` calls in a hidden `finally`. It solves two problems: the boilerplate of correct nested cleanup, which almost nobody wrote properly, and exception masking — in a hand-written `finally`, an exception from `close()` replaces the body's exception and destroys the real diagnosis.

> **Q: "In what order are resources closed?"**
> A: Reverse order of declaration — last opened, first closed. That is correct for

and the `Connection`.

Q: "What is a suppressed exception?"

A: If the try body throws and `close()` also throws, the body's exception propagates and `close()`'s is attached to it via `addSuppressed`. You read them with `getSuppressed()`, and stack traces print them under a "Suppressed:" heading. It exists so that neither exception is lost, which is exactly what the old manual pattern got wrong.

Q: "`AutoCloseable` or `Closeable`?"

A: `AutoCloseable` is the Java 7 interface try-with-resources works with, and its `close()` declares `throws Exception` so anything can implement it. `Closeable` is the older `java.io` interface, now extending `AutoCloseable`, narrowing the declaration to `IOException` and documenting `close()` as idempotent. Implement `AutoCloseable` for a general resource; you get `Closeable` for free with I/O types.

Q: "What is a resource you might forget to close?"

A: `Files.lines()` and `Files.walk()` — they return streams backed by an open file handle. Because almost no other stream needs closing, people use them like an ordinary stream and leak descriptors until GC runs, which on a busy server shows up as "too many open files". Both must be used inside try-with-resources.

Q: "Can you use a variable declared outside the try?"

A: Since Java 9, yes — `try (existingResource) { … }` works as long as the variable is final or effectively final. Before that you had to reassign it to a fresh local inside the parentheses, which was pure ceremony.

- `try (Resource r = …) { }` → `close()` generated in a hidden `finally`.
- Closed in **reverse declaration order**.
- Body exception **wins**; `close()`'s is attached via `addSuppressed()`.
- Resource must implement `AutoCloseable` (`Closeable` = I/O, `IOException`, idempotent).
- Java 9+: `try (effectivelyFinalVar) { }` — no re-declaration.
- Make your `close()` **idempotent** and non-throwing where possible.
- ⚠ `Files.lines()` / `Files.walk()` hold file handles — always close them.
- Never hand-write the `finally` version again.

QUICK CHECK — no marks, just for you.

1. In what order are resources declared in a try-with-resources statement closed?

- ◯ Alphabetically by variable name
- ◯ Reverse order of declaration — last opened, first closed
- ◯ Declaration order
- ◯ Unspecified

Check answer

2. The try body throws and close() also throws. Which exception reaches the caller?

- ◯ Only the first one thrown, with the other discarded
- ◯ The one from close(), because it happened last
- ◯ The one from the body — close()'s is attached to it as a suppressed exception, retrievable with getSuppressed()

Both are wrapped in a new CompositeException

Check answer

3. Which of these must be closed in a try-with-resources, despite looking like an ordinary stream?

- ○ list.stream()
- ○ IntStream.range(0, 10)
- ○ Stream.of("a", "b")
- ○ Files.lines(path) — it is backed by an open file handle

Check answer

‹ Previous
try, catch, finally and Control Flow

Next ›
Designing Custom Exceptions

LEARN MORE

ARTICLES & DOCS

The try-with-resources statement
docs.oracle.com

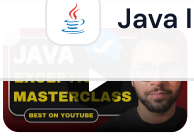
AutoCloseable — API docs
docs.oracle.com

VIDEOS



Try with Resources with Catch and Finally | Java Exception handling
Ram N Java

Java Interview Kit



Master Exceptions in Java: Try, Catch, Finally, Throw, Throws, try-with-resources & Custom Exception

Engineering Digest

Hand-picked references for this lesson.